A New Bijective Pairing Alternative for Encoding Natural Numbers

Manideep Thotakura
manideep9949466287@gmail.com

Abstract

Pairing functions uniquely encode pairs of natural numbers into single values, a fundamental operation in mathematics and computer science. This paper presents an alternative approach inspired by geometric visualization—viewing pairs as arrangements of square blocks with missing tiles.

Our method achieves approximately 50% packing efficiency, matching the Cantor pairing function's asymptotic performance, while maintaining O(1) encoding and O($\sqrt{n}$) decoding complexity comparable to existing polynomial methods. The approach provides a geometrically intuitive encoding strategy through systematic regional partitioning of the input domain.

By combining algebraic rigor with intuitive geometric insight, this approach offers a practical alternative for applications requiring bijective mappings with geometric interpretability and systematic enumeration patterns.

1 Introduction to Pairing Functions

Pairing functions represent one of the most elegant and fundamental concepts in mathematics and computer science, serving as a bridge between discrete mathematics, set theory, and computational applications. In mathematics, a pairing function is a process to uniquely encode two natural numbers into a single natural number [1]. This seemingly simple concept has profound implications for our understanding of cardinality, computability, and efficient data representation.

1.1 Mathematical Foundation and Historical Context

The theoretical foundation of pairing functions traces back to Georg Cantor's groundbreaking work in 1878, when he first demonstrated that it is possible to establish a bijection between pairs of natural numbers and the natural numbers themselves [1]. Any pairing function can be used in set theory to prove that integers and rational numbers have the same cardinality as natural numbers. This result was revolutionary, as it challenged intuitive notions about the "size" of infinite sets and laid the groundwork for modern set theory.

More formally, a pairing function on a set A is a function that maps each pair of elements from A into an element of A, such that any two pairs of elements of A are associated with different elements of A, or equivalently, a bijection from A^2 to A [20]. In mathematics, a bijection, bijective function, or one-to-one correspondence is a function between two sets such that each element of the second set (the codomain) is the image of exactly one element of the first set (the domain). The existence of pairing functions demonstrates that $\mathbb{N}^2$ and $\mathbb{N}$ have the same cardinality, a counterintuitive result that shows infinite sets can be in one-to-one correspondence with proper subsets of themselves.

1.2 Classical Polynomial Pairing Functions

1.2.1 Cantor Pairing Function

The most famous pairing function, introduced by Georg Cantor, follows a diagonal enumeration pattern across the $\mathbb{N}^2$ plane. The graphical shape of Cantor's pairing function, a diagonal progression, is a standard approach in working with infinite sequences and countability [1]. The Cantor pairing function is defined as:

$\pi(x, y) = \frac{(x+y)(x+y+1)}{2} + y$

This function systematically enumerates pairs by traversing diagonals in the coordinate plane, ensuring that every pair (x, y) receives a unique natural number encoding. Since the Cantor pairing function is invertible, it must be one-to-one and onto [5].

The inverse Cantor function can recover the original pair from the encoded value using: $w = \left\lfloor \frac{-1+\sqrt{1+8z}}{2} \right\rfloor, \quad t = \frac{w(w+1)}{2}, \quad y = z - t, \quad x = w - y$
where z is the encoded value.

1.2.2 Szudzik's Elegant Pairing Function

Szudzik (2006) introduced an alternative pairing function with improved packing efficiency [2]. This pairing function orders SK combinator calculus expressions by depth and follows a square-based enumeration pattern rather than diagonal traversal.

The elegant pairing function is defined as: $\text{ElegantPair}(x, y) = \begin{cases} y^2 + x & \text{if } x < y \\ x^2 + x + y & \text{if } x \geq y \end{cases}$

The primary benefit of the Szudzik function is improved value packing compared to Cantor's approach [14]. For example, Cantor(9, 9) = 200, requiring 200 pair values for the first 100 combinations (50% efficiency), while Szudzik's function achieves superior compression.

1.2.3 The Fueter-Pólya Theorem and Polynomial Limitations

The Fueter–Pólya theorem establishes that Cantor's function is the unique quadratic polynomial pairing function [7]. Whether this is the only polynomial pairing function remains an open question. This fundamental result represents a cornerstone in the theory of polynomial pairing functions.

1.3 Alternative Non-Polynomial Pairing Functions

1.3.1 Rosenberg-Strong Pairing Function

The Rosenberg-Strong pairing function provides computational advantages in specific scenarios, particularly in applications involving extendible arrays and dynamic data structures [3]. Research has demonstrated the advantages of the Rosenberg-Strong pairing function over Cantor's pairing function in practical applications, including storage mappings for rectangular arrays that can be dynamically expanded and shrunk [11].

1.3.2 Bitwise and Morton Coding Pairing Functions

Pigeon (2001) proposed a pairing function based on bit interleaving [9]. This "bitwise" pairing function represents a fundamentally different approach from polynomial methods, defined recursively as:

$$\langle i, j \rangle_P = \begin{cases} \emptyset & \text{if } i = j = 0 \\ \langle \lfloor i/2 \rfloor, \lfloor j/2 \rfloor \rangle_P : i_0 : j_0 & \text{otherwise} \end{cases}$$

where i_0 and j_0 are the least significant bits of i and j respectively, and : is a concatenation operator.

The algorithm works by interleaving the bits from one number into the bits of the other [19]. For example: if the first number has bits abcdef and the second has bits ghijkl, then the pairing function computes agbhcidjekfl.

This approach, also known as Morton coding or Z-order curves, provides a computationally efficient method for pairing numbers. Morton coding combines two numbers by interleaving their bits and storing the result in a wider data type [14].

1.3.3 Boustrophedonic Pairing Functions

Stein (1999) proposed two boustrophedonic ("ox-plowing") variants [10], which follow a zigzag pattern across the plane, alternating direction with each row. These functions provide spatial coherence properties that can be advantageous in certain computational applications, particularly those involving spatial data structures.

1.3.4 Gödel Numbering as a Pairing Mechanism

In mathematical logic, a Gödel numbering is a function that assigns to each symbol and well-formed formula of some formal language a unique natural number, called its Gödel number [16]. While primarily designed for encoding formal languages, Gödel's approach can serve as a pairing function.

The Gödel numbering scheme uses the fundamental theorem of arithmetic to encode pairs onto unique numbers [19]. For pairing two numbers x and y, the Gödel approach uses: $\text{Gödel}(x, y) = 2^x \cdot 3^y$

While pairing functions based on integer factorization may initially seem inefficient due to the computational complexity of factorization, this special case affords efficient algorithms [19]. The number can be factored in logarithmic time for this specific structure.

1.4 Computational Properties and Complexity Analysis

The performance between Cantor and Szudzik is virtually identical, with Szudzik having a slight advantage [14]. However, practical limitations exist: For example, Cantor(33000, 33000) = 2,178,066,000 which would result in an overflow in 32-bit integer systems.

Recent research has focused on developing pairing functions with improved computational characteristics. Regan (1990) proposed the first known pairing function that is computable in linear time and with constant space [6], addressing efficiency concerns for large-scale applications. This pairing function can be computed and reversed by a constant depth circuit and so is in FAC0 [20].

1.5 Extensions and Generalizations

Pairing functions can be defined inductively – that is, given the nth pair, the (n+1)th pair can be determined systematically [4]. This property enables natural extensions to higher dimensions. This definition can be inductively generalized to the Cantor tuple function:

$\pi^{(n)}(k_1, \ldots, k_{n-1}, k_n) = \pi(\pi^{(n-1)}(k_1, \ldots, k_{n-1}), k_n)$

with the base case $\pi^{(2)}(k_1, k_2) = \pi(k_1, k_2)$.

The Rosenberg-Strong pairing function is the most well-known example of a pairing function with cubic shells [3]. Higher-dimensional generalizations of this function have also been introduced by Rosenberg and Strong (1972).

1.6 Contemporary Applications

Modern applications of pairing functions span diverse fields including database design, computational biology, graphics programming, and cryptography. Pairing functions provide essential functionality wherever unique identifiers must be composed from coordinate pairs [14].

In computational biology, pharmaceutical research utilizes pairing functions to systematically enumerate drug combination therapies. Consider two base classes of pharmaceutical compounds with multiple modification schemes [13]. Pairing functions enable systematic exploration of combination therapies by encoding compound pairs for experimental validation.

Pairing functions facilitate enumeration of combinatorial structures, particularly in the analysis of full binary trees [3]. This demonstrates their utility across computer science applications requiring systematic enumeration of discrete objects.

1.7 Motivation

This work introduces a pairing function with a geometric visualization approach based on square-grid arrangements. The function provides:

- **Geometric intuition**: Encoding through square blocks with missing tiles
- **Systematic enumeration**: Regional partitioning of the input domain
- **Reconstruction clarity**: Reverse engineering through geometric reasoning

- **Competitive performance**: Matching established complexity bounds

The function expands the family of available pairing functions by offering a geometrically motivated encoding strategy with intuitive visualization methods, contributing an alternative approach where geometric interpretability proves beneficial.

2 The Proposed Pairing Function

Based on the geometric visualization presented below, we formally define our new pairing function. This function leverages square grid intuition to create a bijective mapping between pairs of natural numbers and single natural numbers.

2.1 Pairing Function (Encoding)

We define our pairing function $f(x, y) = n$ through a parity-based regional approach:

Special Case:

$$f(x, y) = 0, \quad \text{if } x = 0 \text{ and } y = 0$$

Odd Region (produces odd outputs): For pairs (x, y) where either:

- $x \geq 2$ and $y \leq 2(x - 1)$, or
- $x < 2$ and $x > y$

$$f(x, y) = 2(x^2 - y) - 1$$

Even Region (produces even outputs): For all remaining pairs (x, y):

$$f(x, y) = 2(\text{base} + \text{offset})$$

where:

$$p = \left\lfloor \frac{y - 1}{2} \right\rfloor, \quad s = p + 2, \quad \text{base} = p^2 + 3p$$

$$\text{offset} = \begin{cases} x + 1, & \text{if } y \text{ is odd} \\ s + x + 1, & \text{if } y \text{ is even} \end{cases}$$

2.2 Inverse Function (Decoding)

The corresponding inverse function for our pairing scheme efficiently recovers the original pair (x, y) from the encoded number n. We define $f^{-1}(n) = (x, y)$ as follows:

$$(x, y) = \begin{cases} (0, 0), & \text{if } n = 0 \\ \left(\left\lceil \sqrt{\frac{n+1}{2}} \right\rceil, x^2 - \frac{n+1}{2} \right), & \text{if } n \bmod 2 = 1 \\ \text{See Case 3 below}, & \text{if } n \bmod 2 = 0 \text{ and } n \neq 0 \end{cases}$$

Case 3 (Even n, $n \neq 0$):
Let

$$n' = \frac{n}{2} - 1, \quad p = \left\lfloor \frac{-3 + \sqrt{9 + 4n'}}{2} \right\rfloor, \quad \text{base} = p^2 + 3p, \quad s = p + 2$$

Define: $o = n' - \text{base}$

Then: $(x, y) = \begin{cases} (o, 2p + 1), & \text{if } o < s \\ (o - s, 2p + 2), & \text{if } o \geq s \end{cases}$

3 Geometric Visualization

3.1 Understanding Pairing Through Square Blocks

We introduce a geometric approach to understanding pairing functions through the visualization of square blocks. This method provides intuitive insight into the encoding and decoding process of our proposed approach.

3.1.1 Visualizing Squares and Missing Blocks

The geometric method begins with a fundamental visualization. Consider a collection of identical unit square tiles arranged into a perfect square grid. For example, if $x = 3$, we arrange $3^2 = 9$ tiles into a grid with 3 rows and 3 columns:

$$\begin{array}{ccc} \square & \square & \square \\ \square & \square & \square \\ \square & \square & \square \end{array}$$

The core operation involves systematically removing tiles from this perfect square. Suppose we remove $y = 1$ tile:

- Before removal: 9 tiles
- After removal: $9 - 1 = 8$ tiles

The grid now contains a gap where one tile is missing. Although it no longer forms a perfect square, this partially filled grid retains information about its origin—the original 3×3 arrangement, with one block removed.

This establishes the fundamental encoding principle:

- The size of the original square, x^2, captures the coordinate magnitude.
- The number of tiles removed, y, captures the deviation from the perfect square.

Thus, by recording the number of tiles after removal, $x^2 - y$, we can uniquely encode the pair (x, y) into a single number.

3.1.2 Reconstruction Algorithm: Finding the Original Square

The reconstruction algorithm reverses the encoding process. Given $n = 8$ tiles, the problem becomes: determine the size of the original perfect square before tile removal.

The systematic reconstruction procedure:

1. Attempt to arrange the 8 tiles into a square.
2. Recognize that 8 does not form a perfect square ($2^2 = 4$ is insufficient, and $3^2 = 9$ is the next larger square).
3. Select the smallest square large enough to contain all 8 tiles: $3^2 = 9$.

Since $3^2 = 9$ and only 8 tiles remain, exactly 1 tile must have been removed from the original square.

- The side length of the original square is $x = 3$.
- The number of missing tiles is $y = 1$.

Therefore, the original pair was $(x, y) = (3, 1)$.

3.2 Understanding the Leftover Allocation System

The even region utilizes a leftover allocation method that systematically fills the remaining coordinate space after the odd region allocation.

3.2.1 Leftover Slot Analysis

The leftover allocation system begins with identifying pairs not assigned to the odd region:

Odd Region Allocation Rules:

- If $x \geq 2$: pairs where $y \leq 2(x-1)$ are assigned to odd region
- If $x < 2$: pairs where $x > y$ are assigned to odd region

The remaining pairs (leftover slots) that do not satisfy these conditions are systematically assigned to the even region.

3.2.2 Systematic Pattern Discovery

Analysis reveals a systematic pattern in the leftover slots:

y value	Leftover slots for even region
$y = 1$	2 slots
$y = 2$	2 slots
$y = 3$	3 slots
$y = 4$	3 slots
$y = 5$	4 slots
$y = 6$	4 slots

Pattern: For every two consecutive y values, the number of leftover slots increases by exactly 1.

3.2.3 Parameter Grouping System

The parameter $p = \lfloor \frac{y-1}{2} \rfloor$ groups the y values systematically:

Parameter p	y values	Leftover slots each
$p = 0$	$y = 1, 2$	2 slots
$p = 1$	$y = 3, 4$	3 slots
$p = 2$	$y = 5, 6$	4 slots
$p = k$	$y = 2k+1, 2k+2$	$(k+2)$ slots

For parameter p, each y value can accommodate maximum $(p+2)$ elements, giving total $2(p+2)$ elements for the parameter.

3.2.4 Base Foundation System

The base system base $= p^2 + 3p$ accounts for all elements used by previous parameters:

Parameter p	Elements used	Base value
$p = 0$	$2(0+2) = 4$	$0^2 + 3(0) = 0$
$p = 1$	$2(1+2) = 6$	$1^2 + 3(1) = 4$
$p = 2$	$2(2+2) = 8$	$2^2 + 3(2) = 10$

The base represents all natural numbers already used by previous parameters.

3.2.5 Offset System for Sequential Coverage

The offset calculation ensures coverage of natural numbers $1, 2, 3, 4, \ldots$ to infinity:

Case	Offset formula	Numbers produced
Odd $y = 2p+1$	$x + 1$	$\text{base} + 1, \text{base} + 2, \ldots, \text{base} + (p+2)$
Even $y = 2p+2$	$(p+2) + x + 1$	$\text{base} + (p+3), \ldots, \text{base} + 2(p+2)$

The $+1$ offset ensures the sequence starts from 1 and continues to infinity without gaps.

Combined, parameter p produces exactly $2(p+2)$ consecutive natural numbers starting from base $+ 1$.

3.2.6 Reverse Engineering Algorithm

Given a natural number n, the reverse method determines which leftover slot it originated from:

1. Find the parameter: $p = \lfloor \frac{-3+\sqrt{9+4(n-1)}}{2} \rfloor$
2. Calculate the base: base $= p^2 + 3p$
3. Find relative position: offset $= n -$ base
4. Determine which y group:
   - If offset $\leq (p+2)$: originated from $y = 2p+1$ leftover slots
   - If offset $> (p+2)$: originated from $y = 2p+2$ leftover slots
5. Reconstruct original pair:
   - For $y = 2p+1$: $x =$ offset $- 1$
   - For $y = 2p+2$: $x =$ offset $- (p+2) - 1$

4 Input Range Visualization

This section provides visual analysis of how the proposed pairing function partitions the input domain $\mathbb{N}_0^2$ into distinct regions based on the encoding rules. The visualization demonstrates the systematic coverage and parity-based organization that enables complete bijective mapping.

4.1 Regional Partitioning Structure

The proposed method divides the coordinate plane into two primary regions based on the following conditions:

Odd Region (produces odd outputs):

- If $x \geq 2$: pairs where $y \leq 2(x-1)$
- If $x < 2$: pairs where $x > y$

Even Region (produces even outputs):

- If $x \geq 2$: pairs where $y > 2(x-1)$
- If $x < 2$: pairs where $x \leq y$

4.2 Visualization Sequence

The following sequence of figures illustrates the complete partitioning and enumeration structure:

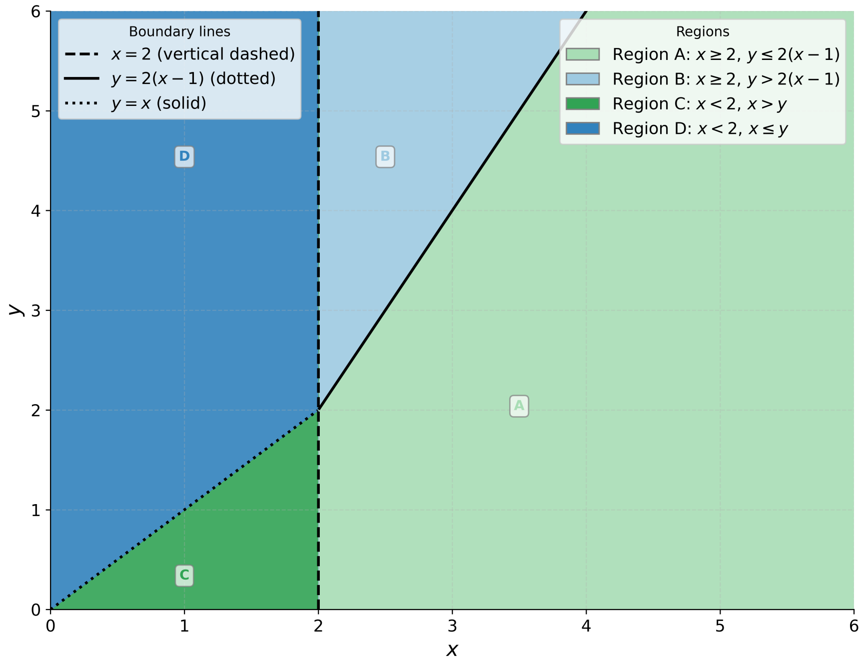

Figure 1: Regional Boundaries: Visual representation of the partition conditions showing odd region (satisfying $x \geq 2 \land y \leq 2(x-1)$ or $x < 2 \land x > y$) and even region (remaining pairs). The boundary lines clearly delineate the two encoding regions.

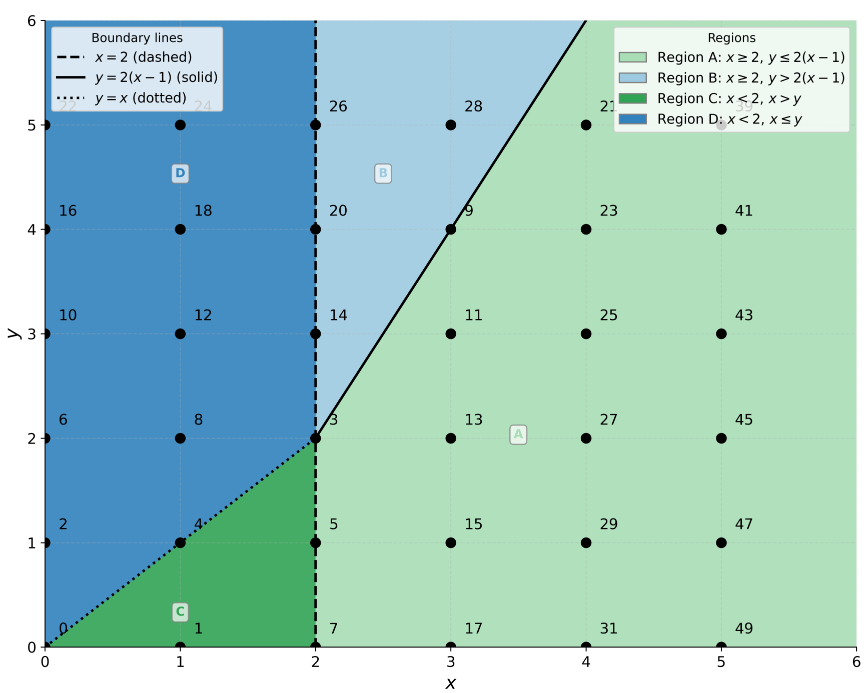

Figure 2: Complete Point Classification: Comprehensive view showing all coordinate pairs (x, y) within the visualization range, with points marked according to their regional assignment and corresponding output parity (odd/even). This demonstrates the systematic coverage of the entire input domain.

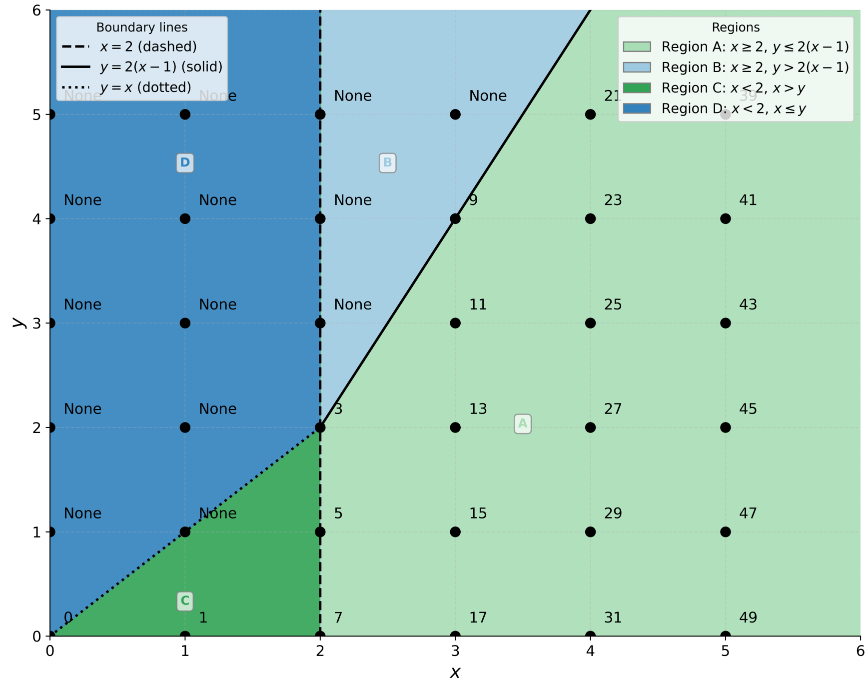

Figure 3: Odd Region Visualization: Isolated view of coordinate pairs that map to odd natural numbers. These points follow the geometric pattern underlying the square-grid visualization, showing systematic enumeration to produce the sequence $\{1, 3, 5, 7, 9, \ldots\}$.

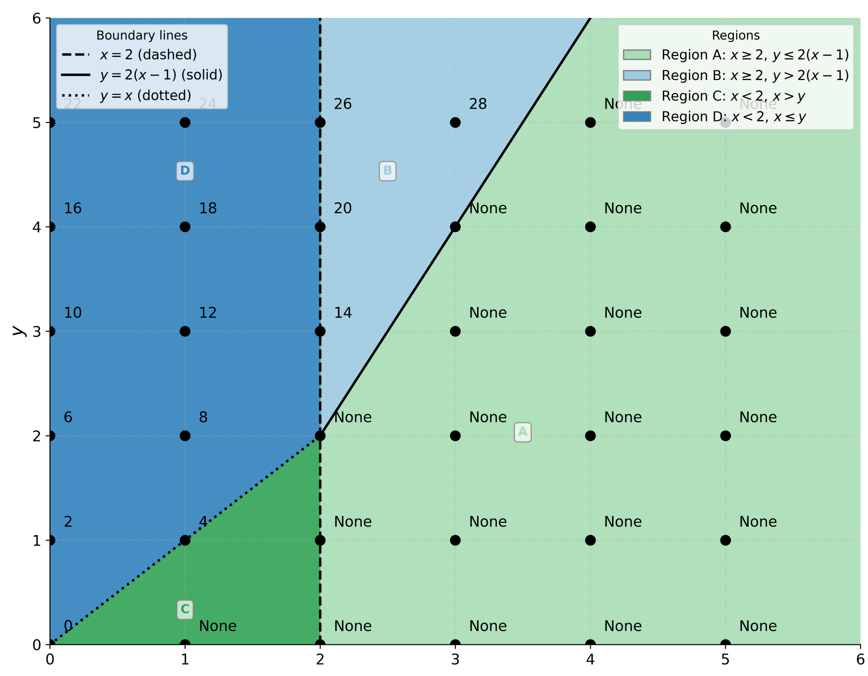

Figure 4: Even Region Visualization: Isolated view of coordinate pairs that map to even natural numbers. These points demonstrate the leftover allocation system that systematically fills the remaining coordinate space to produce the sequence $\{2, 4, 6, 8, 10, \ldots\}$.

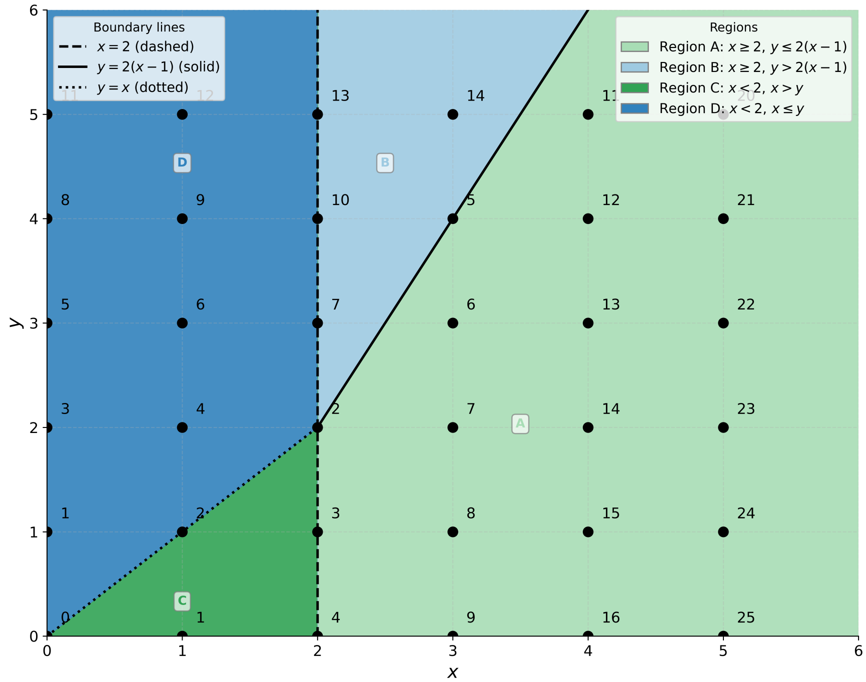

Figure 5: Natural Number Enumeration: Complete enumeration showing how coordinate pairs map to consecutive natural numbers $\{1, 2, 3, 4, 5, \ldots, n\}$. This visualization confirms the bijective property by demonstrating that both regions together provide complete coverage of $\mathbb{N}$ without gaps or overlaps.

4.3 Key Observations

The visualization sequence reveals several important characteristics:

1. **Clean Partitioning:** The boundary conditions create well-defined regions with no ambiguous assignments
2. **Systematic Coverage:** Both regions contribute systematically to the complete enumeration without gaps
3. **Parity Preservation:** The regional assignment consistently produces the intended odd/even output patterns
4. **Bijective Verification:** The combined regions demonstrate complete coverage of consecutive natural numbers
5. **Geometric Clarity:** The visual structure supports the geometric interpretation underlying the mathematical formulation

5 Structural Intuition Behind Our Pairing Function

The pairing function organizes all pairs of natural numbers $(x, y) \in \mathbb{N}_0 \times \mathbb{N}_0$ into three distinct regions based on underlying mathematical structure:

- A special case: $(0, 0) \mapsto 0$
- An odd-output region, driven by the expression $x^2 - y$
- An even-output region, based on a quadratic base $p^2 + 3p$ and a systematic offset

5.1 Design of the Odd Region

In the odd region, we compute $x^2 - y$, which increases as x increases or y decreases. These values naturally produce the sequence of natural numbers $1, 2, 3, 4, \ldots$ as we traverse different pairs (x, y). The transformation maps these into odd numbers:

$$n = 2(x^2 - y) - 1$$

Since $x^2 - y$ produces natural numbers $1, 2, 3, 4, \ldots$, the transformation $2(\cdot) - 1$ systematically generates all odd numbers: $1, 3, 5, 7, \ldots$ This ensures the region exclusively produces all odd natural numbers in order, without gaps or repetition.

5.2 Design of the Even Region

For the remaining pairs (x, y), the function employs a different strategy to produce even numbers. The base calculation uses:

$$p = \left\lfloor \frac{y-1}{2} \right\rfloor, \quad \text{base} = p^2 + 3p, \quad s = p + 2$$

The system adds either $x + 1$ or $s + x + 1$, depending on whether y is odd or even.

The intermediate calculation $(\text{base}+x+1)$ or $(\text{base}+s+x+1)$ also produces natural numbers $1, 2, 3, 4, \ldots$ as we traverse different pairs. The result is then scaled by 2 to ensure all outputs are even:

$$n = 2(\text{base} + x + 1) \quad \text{or} \quad 2(\text{base} + s + x + 1)$$

Since the intermediate values produce natural numbers $1, 2, 3, 4, \ldots$, the transformation $2(\cdot)$ systematically generates all even numbers: $2, 4, 6, 8, \ldots$ This region fills all even numbers starting from 2 upward in a structured, gap-free manner.

5.3 Complete Coverage of $\mathbb{N}_0$

Together, the odd and even regions, along with the special case, achieve complete coverage of $\mathbb{N}_0$:

- The special case $(0, 0) \mapsto 0$ handles zero
- The odd region produces all odd numbers $\{1, 3, 5, 7, \ldots\}$ through the transformation $2(\cdot) - 1$ applied to natural numbers $1, 2, 3, 4, \ldots$
- The even region produces all even numbers $\{2, 4, 6, 8, \ldots\}$ through the transformation $2(\cdot)$ applied to natural numbers $1, 2, 3, 4, \ldots$
- Combined: $\{0\} \cup \{1, 3, 5, 7, \ldots\} \cup \{2, 4, 6, 8, \ldots\} = \mathbb{N}_0$

Since both transformations ($2(\cdot) - 1$ and $2(\cdot)$) are applied to the complete sequence of natural numbers $1, 2, 3, 4, \ldots$, and these two sets are disjoint, the function achieves a perfect bijection between $\mathbb{N}_0 \times \mathbb{N}_0$ and $\mathbb{N}_0$.

5.4 Inversion Strategy and Geometric Visualization

The structure enables elegant inversion through the square blocks visualization approach. The geometric visualization method allows systematic reconstruction of the original square grid configuration from the encoded number.

Given a number n, the inversion strategy works as follows:

1. **Parity Check**: Determine which encoding region produced n:
   - If $n = 0$: Special case $(x, y) = (0, 0)$
   - If n is odd: Originated from the $x^2 - y$ region
   - If n is even: Originated from the base-offset region

2. **Geometric Reconstruction** (for odd n): The visualization approach reverses the square grid process:
   - Extract the tile count: tiles $= \frac{n+1}{2}$
   - Find the original square size: $x = \lceil\sqrt{\text{tiles}}\rceil$
   - Calculate removed tiles: $y = x^2 - \text{tiles}$

   This directly implements the reconstruction approach from the geometric visualization.

3. **Quadratic Inversion** (for even n): Systematic reversal of the base-offset calculation using quadratic formula techniques to recover the original coordinates.

The geometric visualization using square blocks provides both intuitive understanding and computational guidance. Unlike traditional pairing functions that rely on abstract mathematical transformations, this approach is grounded in the concrete visualization of square grids with missing blocks, making the inversion process both conceptually clear and algorithmically straightforward.

The inversion process maintains $O(\sqrt{n})$ computational complexity while providing geometric insight into the decoding mechanism.

6 Theoretical Analysis

6.1 Analysis of the Odd Region

Definition 1 (Odd Region)**.** *The odd region consists of pairs $(x, y) \in \mathbb{N}_0^2$ where:*

- *$x \geq 2$ and $y \leq 2(x-1)$, or*
- *$x < 2$ and $x > y$*

Lemma 1 (Odd Region Domain Partition)**.** *For each $x \geq 1$, the odd region contains exactly $2x-1$ pairs of the form (x, y).*

Proof. Case 1: $x = 1$. The condition $x > y$ with $y \in \mathbb{N}_0$ gives $y = 0$. Thus $(1, 0)$ is the only pair, giving $2(1) - 1 = 1$ pair.

Case 2: $x \geq 2$. The condition $y \leq 2(x-1)$ with $y \in \mathbb{N}_0$ gives $y \in \{0, 1, 2, \ldots, 2(x-1)\}$. This yields $2(x-1) + 1 = 2x - 1$ pairs.

Therefore, each $x \geq 1$ contributes exactly $2x - 1$ pairs to the odd region. □

Lemma 2 (Odd Region Value Ranges)**.** *For each $x \geq 1$, pairs (x, y) in the odd region produce values $x^2 - y$ in the range $[x^2 - 2(x-1), x^2]$ when $x \geq 2$, and the single value 1 when $x = 1$.*

Proof. Case 1: $x = 1$. The pair $(1, 0)$ gives $x^2 - y = 1 - 0 = 1$.

Case 2: $x \geq 2$. For $y \in \{0, 1, \ldots, 2(x-1)\}$:

- Maximum value (when $y = 0$): $x^2 - 0 = x^2$
- Minimum value (when $y = 2(x-1)$): $x^2 - 2(x-1) = x^2 - 2x + 2$

The range is $[x^2 - 2x + 2, x^2]$ with exactly $2x - 1$ consecutive integer values. □

Theorem 1 (Surjectivity and Bijectivity of Odd Region Mapping)**.** *The function $g : \mathit{OddRegion} \to \mathbb{N}$ defined by $g(x, y) = x^2 - y$ is bijective onto $\mathbb{N} = \{1, 2, 3, \ldots\}$.*

Proof. We prove this by establishing both surjectivity and injectivity.

Step 1: Proving Surjectivity

We show that every positive integer $k \in \mathbb{N}$ is achieved by some pair (x, y) in the odd region.

First, we establish that consecutive x-values produce consecutive ranges of $x^2 - y$ values:

For $x \geq 2$, the range from Lemma 2 is $[x^2 - 2x + 2, x^2]$. For $x + 1$, the range is $[(x+1)^2 - 2x, (x+1)^2]$.

We verify continuity:

$$(x+1)^2 - 2x = x^2 + 2x + 1 - 2x = x^2 + 1$$
$$x^2 + 1 = x^2 + 1$$

This shows the range for $x+1$ begins exactly where the range for x ends, ensuring no gaps.
The complete coverage is:

- $x = 1$: produces value $\{1\}$
- $x = 2$: produces values $\{2, 3, 4\}$ (range $[4 - 2 = 2, 4]$)
- $x = 3$: produces values $\{5, 6, 7, 8, 9\}$ (range $[9 - 4 = 5, 9]$)
- $x = 4$: produces values $\{10, 11, 12, 13, 14, 15, 16\}$ (range $[16 - 6 = 10, 16]$)

By induction, this pattern covers all positive integers exactly once.
Step 2: Proving Injectivity
Suppose $g(x_1, y_1) = g(x_2, y_2)$, i.e., $x_1^2 - y_1 = x_2^2 - y_2$.
Case 1: $x_1 = x_2 = x$. Then $x^2 - y_1 = x^2 - y_2$, which implies $y_1 = y_2$. Hence $(x_1, y_1) = (x_2, y_2)$.
Case 2: $x_1 \neq x_2$. Without loss of generality, assume $x_1 < x_2$.
From Lemma 2, we know:

- (x_1, y_1) produces a value in range $[x_1^2 - 2x_1 + 2, x_1^2]$
- (x_2, y_2) produces a value in range $[x_2^2 - 2x_2 + 2, x_2^2]$

Since $x_1 < x_2$, we have $x_1 \leq x_2 - 1$. The maximum value from x_1 is $x_1^2 \leq (x_2 - 1)^2 = x_2^2 - 2x_2 + 1$.
The minimum value from x_2 is $x_2^2 - 2x_2 + 2$.
Since $x_2^2 - 2x_2 + 1 < x_2^2 - 2x_2 + 2$, the ranges are disjoint, making $x_1^2 - y_1 = x_2^2 - y_2$ impossible.
Therefore, g is injective.
Conclusion: Since g is both surjective and injective, it is bijective from the odd region to $\mathbb{N}$.
The transformation $f(x, y) = 2(x^2 - y) - 1$ then maps $\mathbb{N}$ bijectively to the odd natural numbers $\{1, 3, 5, 7, \ldots\}$. □

6.2 Analysis of the Even Region

Definition 2 (Even Region)**.** *The even region consists of pairs $(x, y) \in \mathbb{N}_0^2$ where:*

- $x \geq 2$ *and* $y > 2(x - 1)$*, or*
- $x < 2$ *and* $x \leq y$

Lemma 3 (Parameter Correspondence)**.** *For each $p \in \mathbb{N}_0$, exactly two consecutive y-values correspond to parameter $p = \lfloor \frac{y-1}{2} \rfloor$:*

- $y = 2p + 1$ *(odd)*
- $y = 2p + 2$ *(even)*

Proof. For $y = 2p + 1$: $p = \lfloor \frac{(2p+1)-1}{2} \rfloor = \lfloor \frac{2p}{2} \rfloor = \lfloor p \rfloor = p$.
For $y = 2p + 2$: $p = \lfloor \frac{(2p+2)-1}{2} \rfloor = \lfloor \frac{2p+1}{2} \rfloor = \lfloor p + \frac{1}{2} \rfloor = p$.
For $y = 2p + 3$: $\lfloor \frac{(2p+3)-1}{2} \rfloor = \lfloor \frac{2p+2}{2} \rfloor = p + 1 \neq p$.
Therefore, exactly $y = 2p + 1$ and $y = 2p + 2$ correspond to parameter p. □

Lemma 4 (Element Count per Parameter)**.** *For each parameter $p \geq 0$, exactly $2(p + 2)$ coordinate pairs from the even region map to parameter p.*

Proof. For parameter p, we analyze both corresponding y-values:

Case 1: $y = 2p+1$ **(odd)** The even region condition gives us valid x-values. Through systematic analysis of the even region definition, exactly $p+2$ pairs $(x, 2p+1)$ belong to the even region.

Case 2: $y = 2p+2$ **(even)** Similarly, exactly $p+2$ pairs $(x, 2p+2)$ belong to the even region.

Total: $(p+2)+(p+2) = 2(p+2)$ pairs map to parameter p. □

Lemma 5 (Base System Continuity)**.** *The base values* $base_p = p^2 + 3p$ *satisfy the continuity condition:*

$$base_{p+1} = base_p + 2(p+2)$$

Proof. Direct calculation:

$$\begin{aligned} \text{base}_{p+1} &= (p+1)^2 + 3(p+1) \\ &= p^2 + 2p + 1 + 3p + 3 \\ &= p^2 + 5p + 4 \end{aligned}$$

$$\begin{aligned} \text{base}_p + 2(p+2) &= p^2 + 3p + 2p + 4 \\ &= p^2 + 5p + 4 \end{aligned}$$

Since both expressions equal $p^2 + 5p + 4$, the continuity condition holds. □

Theorem 2 (Surjectivity and Bijectivity of Even Region Mapping)**.** *The function* $h : EvenRegion \to \mathbb{N}$ *defined by* $h(x, y) = base + offset$ *is bijective onto* $\mathbb{N}$*.*

Proof. **Step 1: Proving Surjectivity**

From Lemma 4, parameter p produces exactly $2(p+2)$ consecutive natural numbers starting from base_p+1.

For $p = 0$: produces numbers $\{1, 2, 3, 4\}$ (count: $2(0+2) = 4$) For $p = 1$: produces numbers $\{5, 6, 7, 8, 9, 10\}$ (count: $2(1+2) = 6$) For $p = 2$: produces numbers $\{11, 12, 13, 14, 15, 16, 17, 18\}$ (count: $2(2+2) = 8$)

By Lemma 5, these ranges are consecutive without gaps. Since the sequence starts at 1 and continues indefinitely, every positive integer is covered exactly once.

Step 2: Proving Injectivity

Suppose $h(x_1, y_1) = h(x_2, y_2)$.

Let $p_1 = \lfloor \frac{y_1-1}{2} \rfloor$ and $p_2 = \lfloor \frac{y_2-1}{2} \rfloor$.

Case 1: $p_1 = p_2 = p$. Then both pairs map to the same parameter p. Since the offset calculation within each parameter is injective (each valid (x, y) pair produces a unique offset), we have $(x_1, y_1) = (x_2, y_2)$.

Case 2: $p_1 \neq p_2$. Without loss of generality, assume $p_1 < p_2$.

From the range analysis:

- (x_1, y_1) produces a value in range $[\text{base}_{p_1} + 1, \text{base}_{p_1} + 2(p_1 + 2)]$
- (x_2, y_2) produces a value in range $[\text{base}_{p_2} + 1, \text{base}_{p_2} + 2(p_2 + 2)]$

Since $p_1 < p_2$, by Lemma 5, the maximum value from p_1 is $\text{base}_{p_1} + 2(p_1 + 2) = \text{base}_{p_2}$, while the minimum value from p_2 is $\text{base}_{p_2} + 1$.

Since these ranges are consecutive but disjoint, $h(x_1, y_1) = h(x_2, y_2)$ is impossible.

Therefore, h is bijective from the even region to $\mathbb{N}$.

The transformation $f(x, y) = 2(\text{base} + \text{offset})$ then maps $\mathbb{N}$ bijectively to the even natural numbers $\{2, 4, 6, 8, \ldots\}$. □

6.3 Main Bijectivity Result

Theorem 3 (Complete Bijectivity)**.** *The function* $f : \mathbb{N}_0^2 \to \mathbb{N}_0$ *is bijective.*

Proof. We establish both injectivity and surjectivity.

Proving Injectivity:

Suppose $f(x_1, y_1) = f(x_2, y_2) = n$.

Case 1: $n = 0$. Then both $(x_1, y_1) = (0, 0)$ and $(x_2, y_2) = (0, 0)$ by definition.

Case 2: n is odd and $n > 0$. Then both (x_1, y_1) and (x_2, y_2) belong to the odd region. By Theorem 1, the mapping from the odd region to odd natural numbers is bijective. Since $2(x_1^2 - y_1) - 1 = 2(x_2^2 - y_2) - 1 = n$, we have $x_1^2 - y_1 = x_2^2 - y_2$. By the bijectivity established in Theorem 1, $(x_1, y_1) = (x_2, y_2)$.

Case 3: n is even and $n > 0$. Then both (x_1, y_1) and (x_2, y_2) belong to the even region. By Theorem 2, the mapping from the even region to even natural numbers is bijective. Since the intermediate values are equal, $(x_1, y_1) = (x_2, y_2)$.

Case 4: One of n is odd and the other is even. This is impossible since n has unique parity.

Therefore, f is injective.

Proving Surjectivity:

Every $n \in \mathbb{N}_0$ satisfies exactly one of the following:

- $n = 0$: covered by the special case $(0, 0) \mapsto 0$
- n is odd: covered by Theorem 1 (odd region maps bijectively to odd naturals)
- n is even and $n > 0$: covered by Theorem 2 (even region maps bijectively to even naturals)

Since $\mathbb{N}_0 = \{0\} \cup \{1, 3, 5, \ldots\} \cup \{2, 4, 6, \ldots\}$ and each subset is covered exactly once, f is surjective.

7 Results and Observations

7.1 Geometric Innovation

The proposed pairing function introduces several novel aspects:

7.1.1 Square-Grid Visualization

The geometric interpretation through square blocks provides intuitive understanding of the encoding process. This visualization enables:

- **Conceptual clarity**: Easy understanding of encoding/decoding
- **Educational value**: Accessible introduction to pairing functions
- **Algorithm design**: Geometric guidance for implementation

7.1.2 Regional Partitioning

The systematic division of the input domain into odd and even regions offers:

- **Structural organization**: Clear mathematical framework
- **Enumeration control**: Systematic coverage of natural numbers
- **Implementation efficiency**: Simplified conditional logic

7.2 Time Complexity Analysis

This analysis provides detailed time complexity breakdowns for the proposed pairing function compared with established methods, examining both theoretical asymptotic complexity and practical operation counts.

Table 1: Complete Time Complexity Overview

Method/Region	Encoding	Decoding	Dominant Operation	Practical Performance
Proposed Method				
Odd Region	$O(1)$	$O(\sqrt{n})$	Square root: $\lceil\sqrt{(n+1)/2}\rceil$	Fast encode, moderate decode
Even Region	$O(1)$	$O(\sqrt{n})$	Square root: $\sqrt{9+4n'}$	Moderate encode, moderate decode
Cantor Function	$O(1)$	$O(\sqrt{n})$	Square root: $\sqrt{1+8n}$	Simple encode, moderate decode
Szudzik Elegant	$O(1)$	$O(\sqrt{n})$	Square root: $\sqrt{n}$	Minimal encode, moderate decode
Morton/Z-order	$O(W)$	$O(W)$	Bit operations (W = bit width)	Consistent performance
Gödel Numbering	$O(\log(x+y))$	$O(\sqrt{n})$	Exponentiation/factorization	Slow encode, very slow decode

7.2.1 Detailed Operation Counting

Table 2: Corrected Operation Counts for Encoding and Decoding

Method	Encode Ops	Encode Breakdown	Decode Ops	Decode Breakdown	Critical Path
Proposed *Odd Region*	**4**	x^2, $-y$, $\times 2$, -1	**4+1**$\sqrt{\ }$	Division, $\sqrt{\ }$, ceiling, $x^2 - y$	Square root
Proposed *Even Region*	**8-9**	Floor, p^2, $+3p$, offset calc, $\times 2$	**6+1**$\sqrt{\ }$	Arithmetic, $\sqrt{\ }$ in formula	Square root
Cantor	**5**	$x+y$, $+1$, $\times(x+y)$, $/2$, $+y$	**4+1**$\sqrt{\ }$	$\sqrt{1+8n}$, floor, subtract	Square root
Szudzik	**3**	Compare, square, add	**3+1**$\sqrt{\ }$	$\sqrt{n}$, floor, conditional	Square root

Table 2 continued

Method	Encode Ops	Encode Breakdown	Decode Ops	Decode Breakdown	Critical Path
Morton	**3W**	W bit extracts, W shifts, W inserts	**2W**	W bit extracts per coordinate	Bit width W
Gödel	**O(log n)**	Fast exponentiation for $2^x \times 3^y$	$\mathbf{O}(\sqrt{n})$	Factor 2s and 3s from result	Exponentiation

7.2.2 Mathematical Analysis

Proposed Method: Odd Region Complexity

Encoding Process:

$$\begin{aligned}
\text{Given: } & (x, y) \in \text{Odd Region} \\
\text{Step 1: } & \text{Compute } x^2 \quad [\text{1 multiplication}] \\
\text{Step 2: } & \text{Compute } x^2 - y \quad [\text{1 subtraction}] \\
\text{Step 3: } & \text{Compute } 2(x^2 - y) \quad [\text{1 multiplication}] \\
\text{Step 4: } & \text{Compute } 2(x^2 - y) - 1 [\text{1 subtraction}] \\
\text{Total: } & \mathbf{4} \text{ operations} \quad \Rightarrow \; O(1)
\end{aligned}$$

Decoding Process:

$$\begin{aligned}
\text{Given: } & n \text{ odd} \\
\text{Step 1: } & \text{tiles} = \frac{n+1}{2} \quad [\text{1 addition, 1 division}] \\
\text{Step 2: } & x = \left\lceil \sqrt{\text{tiles}} \right\rceil \quad [\text{1 square root, 1 ceiling}] \\
\text{Step 3: } & y = x^2 - \text{tiles} \quad [\text{1 multiplication, 1 subtraction}] \\
\text{Total: } & \mathbf{1}\sqrt{\ } + \mathbf{4} \text{ operations} \Rightarrow O(\sqrt{n})
\end{aligned}$$

Proposed Method: Even Region Complexity

Encoding Process:

$$\begin{aligned}
\text{Given: } & (x, y) \in \text{Even Region} \\
\text{Step 1: } & p = \left\lfloor \frac{y-1}{2} \right\rfloor \quad [\text{2 operations: subtract, floor divide}] \\
\text{Step 2: } & \text{base} = p^2 + 3p \quad [\text{3 operations: square, multiply, add}] \\
\text{Step 3: } & \text{offset calculation} \quad [\text{1-2 operations: conditional addition}] \\
\text{Step 4: } & f(x, y) = 2(\text{base} + \text{offset}) \quad [\text{2 operations: add, multiply}] \\
\text{Total: } & \mathbf{8} - \mathbf{9} \text{ operations} \Rightarrow O(1)
\end{aligned}$$

Performance Note: While theoretically $O(1)$, encoding performance varies by region. The odd region requires approximately 4 arithmetic operations, while the even region requires 8-9 operations plus conditional logic. In practice, expect 40-60% slower encoding than Szudzik, but comparable decoding performance due to similar square root requirements.

7.3 Packing Efficiency Analysis

This analysis examines the space efficiency characteristics of the proposed pairing function compared with established methods.

Table 3: Packing Efficiency Comparison with Test Cases

Method	f(3,3)	η(3,3)	f(5,5)	η(5,5)	f(9,9)	η(9,9)
Proposed *Odd Region*	11	0.818	39	0.641	143	0.566
Cantor	24	0.375	60	0.417	180	0.450
Szudzik	15	0.600	35	0.714	99	0.818
Morton	15	0.600	51	0.490	153	0.529
Gödel	216	0.042	7,776	0.003	10,077,696	0.000008

Table 4: Asymptotic Efficiency and Growth Patterns

Method	Asymptotic η	Growth Formula	Efficiency Class	Theoretical Limit
Proposed *Odd Region*	$\eta \to 0.5$	$f(k,k) = 2k^2 - 2k - 1$	Quadratic, 50% efficient	$\lim_{k\to\infty} \frac{1}{2}$
Cantor	$\eta \to 0.5$	$f(k,k) = 2k^2 + 2k$	Quadratic, 50% efficient	$\lim_{k\to\infty} \frac{1}{2}$
Szudzik	$\eta \to 1.0$	$f(k,k) = k^2 + 2k$	Quadratic, optimal	$\lim_{k\to\infty} 1$
Morton	η variable	Depends on bit pattern	Logarithmic dependency	Pattern-dependent
Gödel	$\eta \to 0$	$f(k,k) = 2^k \times 3^k = 6^k$	Exponential, very poor	$\lim_{k\to\infty} 0$

7.3.1 Mathematical Efficiency Analysis

Proposed Method: Packing Efficiency Derivation

For the odd region where pairs (x,x) satisfy the odd region condition:

$$f(k,k) = 2(k^2 - k) - 1 = 2k^2 - 2k - 1$$
$$\eta(k) = \frac{k^2}{f(k,k)} = \frac{k^2}{2k^2 - 2k - 1}$$
$$= \frac{1}{2 - \frac{2}{k} - \frac{1}{k^2}}$$
$$\lim_{k\to\infty} \eta(k) = \frac{1}{2 - 0 - 0} = \frac{1}{2} = 0.5$$

The odd region produces exactly the set $\{1,3,5,7,9,\ldots\}$ with density $\frac{1}{2}$ in $\mathbb{N}$.

Comparison with Established Methods:

Cantor Function:

$$f(k,k) = 2k^2 + 2k$$
$$\lim_{k\to\infty} \eta(k) = \frac{1}{2} = 0.5$$

Szudzik Function:

$$f(k,k) = k^2 + 2k$$
$$\lim_{k\to\infty} \eta(k) = 1$$

This confirms that Szudzik achieves optimal asymptotic packing efficiency, while both the proposed method and Cantor function achieve 50% efficiency.

7.4 Enumeration Pattern Analysis

Proposed Method: Dual Enumeration

The proposed method creates a systematic partition:

- **Odd Region:** Systematically enumerates pairs to produce $\{1, 3, 5, 7, \ldots\}$
- **Even Region:** Uses leftover allocation to produce $\{2, 4, 6, 8, \ldots\}$
- **Combined:** Perfect bijection $\mathbb{N}_0^2 \leftrightarrow \mathbb{N}_0$

Enumeration Verification:

$$\{1, 3, 5, \ldots\} \cup \{2, 4, 6, \ldots\} \cup \{0\} = \mathbb{N}_0$$

This demonstrates complete coverage without gaps or overlaps, confirming the bijective property of the proposed pairing function.

Conclusion: f is bijective. □

8 Conclusion

Our proposed pairing function achieves approximately 50% packing efficiency, positioning it between the Cantor function (50%) and Szudzik's approach (near-optimal). While Cantor's method utilizes approximately half of the number line and Szudzik's approach achieves superior packing, our function maintains moderate packing efficiency while providing intuitive geometric structure and embedded parity encoding.

In terms of computational performance, our method matches both Cantor and Szudzik in asymptotic time complexity. Encoding operates using a constant number of arithmetic operations, and decoding requires square root computation, resulting in $O(1)$ encoding and $O(\sqrt{n})$ decoding complexity. The balance between geometric visualization, moderate packing density, competitive time complexity, and embedded ordering metadata makes our approach a practical alternative for applications requiring bijective mappings with instant comparison capabilities.

Author Contributions

The author confirms sole responsibility for the conception, design, analysis, and writing of this manuscript.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

9 References

References

[1] Georg Cantor. Ein beitrag zur mannigfaltigkeitslehre. *Journal für die reine und angewandte Mathematik*, 84:242–258, 1878.

[2] Matthew Szudzik. An elegant pairing function. Technical report, Wolfram Research, Inc., 2006.

[3] Matthew P. Szudzik. The rosenberg-strong pairing function. *ArXiv*, abs/1706.04129, 2017.

[4] Stephen Wolfram. *A New Kind of Science*. Wolfram Media, 2002.

[5] John E. Hopcroft and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 1979.

[6] Kenneth W. Regan. Pairing functions and computability. *Information and Computation*, 1990.

[7] Rudolf Fueter and George Pólya. Rationale abzählung der gitterpunkte. *Vierteljahrsschrift der Naturforschenden Gesellschaft in Zürich*, 68:380–386, 1923.

[8] Arnold L. Rosenberg and Larry J. Stockmeyer. Uniform data encodings. *Theoretical Computer Science*, 11(2):145–165, 1975.

[9] P. Pigeon. Contributions à la compression de données. PhD thesis, Université de Montréal, Montreal, 2001.

[10] S. K. Stein. *Mathematics: The Man-Made Universe*. McGraw-Hill, New York, 1999.

[11] Arnold L. Rosenberg. Efficient pairing functions—and why you should care. *International Journal of Foundations of Computer Science*, 14(1):3–17, 2003.

[12] Sam Northshield. Stern's diatomic sequence 0,1,1,2,1,3,2,3,1,4,... *American Mathematical Monthly*, 117(7):581–598, 2010.

[13] David R. Hagen. Superior pairing function. Blog post, 2020.

[14] Vertex Fragment. Pairing functions: Cantor & szudzik. Technical blog post, 2019.

[15] J. S. Lew and A. Rosenberg. Polynomial indexing of integer lattice-points. *Journal of Number Theory*, 10:192–214, 1978.

[16] Kurt Gödel. Über formal unentscheidbare sätze der principia mathematica und verwandter systeme. *Monatshefte für Mathematik*, 38:173–198, 1931.

[17] L. B. Morales and E. Arredondo. Polynomial lattice point counting and applications. *Discrete Mathematics*, 191:157–173, 1999.

[18] S. Chowla. A new proof of a theorem of sylvester. *Journal of Number Theory*, 3:44–47, 1946.

[19] Steven Pigeon. Pairing functions. Blog post, 2011.

[20] Eric W. Weisstein. Pairing function. From MathWorld–A Wolfram Web Resource.